

Point-in-Time Cross-Sectional Factor Model

A quantitative equity research project that ranks S&P 500 stocks by predicted next-quarter (3-month) relative return, comparing a linear (Fama-MacBeth-style) baseline against a gradient-boosted (LightGBM) model on the same factor set, and backtesting a long-short decile portfolio built on the resulting rankings.

Built to demonstrate the modeling and data-hygiene practices used in quantitative equity research: point-in-time universe construction, leakage-aware feature engineering, walk-forward validation, and rank-based evaluation — rather than a simplified backtest that overstates performance through survivorship or look-ahead bias.

At a glance

- **Objective:** rank S&P 500 stocks by predicted next-quarter relative return and evaluate whether a gradient-boosted model beats a linear baseline out-of-sample.
- **Stack:** Python, pandas/NumPy, scikit-learn, LightGBM, WRDS/CRSP.
- **Techniques:** point-in-time universe construction (survivorship-bias free), leakage-aware factor engineering, walk-forward cross-validation, rank-based (Information Coefficient) evaluation, long-short decile backtesting with turnover analysis.
- **Status:** pipeline fully implemented and run end-to-end against live WRDS/CRSP data (2012–2026) — see [Results](#).

Results

Produced by a full walk-forward run (60-month rolling train window, 2012–2026, 467–492 point-in-time S&P 500 members per month), predicting **3-month forward relative return** (see Design decisions), evaluated on 36 non-overlapping quarters via [output/model_comparison.csv](#):

Metric	Linear (Fama-MacBeth)	LightGBM
Annualized return	-1.4%	2.8%
Annualized volatility	17.7%	9.4%
Sharpe ratio	-0.08	0.30
Max drawdown	-49.2%	-21.1%
Avg. quarterly IC (Spearman)	-0.011	0.004
Avg. quarterly turnover	0.63	0.75

Honest read of these numbers: stretching the prediction horizon from 1 month to 3 (see Design decisions) is the first change so far that moved the needle in a meaningful direction. LightGBM's IC turns slightly positive (+0.004, vs. -0.004 to -0.011 in every 1-month-horizon variant tried earlier) and its Sharpe improves to 0.30 with a much shallower max drawdown (-21% vs. -63% for the linear model). That IC is still small in absolute terms — this is a weak signal, not a strong one — but it's the first result in this project that's directionally consistent with GBM capturing *some* real nonlinear structure the linear

baseline misses, rather than both models being indistinguishable from noise. The linear baseline remains flat-to-negative (Sharpe -0.08), which is itself a reasonable finding: a purely linear combination of these six price/volume factors doesn't earn a risk-adjusted return on this universe.

Why point-in-time data matters

Universe and prices come from **WRDS/CRSP**, not a simplified source like Yahoo Finance:

- **Point-in-time index membership:** `crsp_a_indexes.dsp500list_v2` gives exact membership spells (start/end dates) for every PERMNO that was ever in the S&P 500 during the sample window — not just today's 500 constituents. This is what actually fixes survivorship bias: the backtest universe on any historical date includes stocks that have since been delisted, acquired, or dropped from the index, because they were genuinely investable at that time.
- **Identifier:** PERMNO, not ticker. Tickers change over time (Facebook → Meta) and get reused after delisting; PERMNO is the identifier CRSP guarantees is stable for the life of a security. A ticker/company-name lookup (`get_permno_ticker_map`) is included only for labeling output for readability — it is never used as a join key in the pipeline itself.
- **Returns:** CRSP's own `ret` field (total return, dividend- and delisting-adjusted by CRSP directly), rather than a return reconstructed from an adjusted-close price series.
- **Market cap:** computed directly from CRSP `prc` × `shrout`, giving a true size factor rather than a liquidity-based stand-in.

Point-in-time filter ordering

Features are computed across each PERMNO's full available price history first, since a stock needs trailing price data to compute momentum correctly even in the months just before or after its actual index membership window. The membership filter is applied afterward, immediately before cross-sectional normalization. This ordering is deliberate:

- Filtering too early would starve trailing-window features (12-month momentum, etc.) of legitimate lookback data right at each stock's entry into the index.
- Filtering too late (e.g. after normalization) would let stocks that weren't actually index members on a given date leak into that date's cross-sectional z-scores and the eventual portfolio.

`features.filter_to_membership` implements this, with the reasoning documented again in its docstring.

Design decisions

- **Momentum window is 12-1, not 12-0** (`mom_12m_ex1` excludes the most recent month) — 1-month reversal is a distinct, opposite-signed effect from 12-month momentum, so mixing them would muddy the signal.
- **Walk-forward validation only**, never a random train/test split — see `model.walk_forward_splits` for the reasoning on purge/embargo gaps as the feature set grows.
- **Forward-return horizon is 3 months, not 1** (`HORIZON = 3` in `main.py`). Monthly stock returns are mostly noise; a 1-month-horizon version of this project (kept in git history) produced IC indistinguishable from zero for both models. This was NOT a drop-in change, because a 3-month-forward target creates two problems a 1-month target doesn't:

1. **Label leakage at the train/test boundary.** A training row's `fwd_ret` isn't actually "known" until `horizon` months after its feature date. `model.run_walk_forward` embargoes the trailing `horizon - 1` months of each training window (via `walk_forward_splits'` `embargo_months`) so no training label depends on data from inside or after the test period.
 2. **Overlapping realized-return windows in the backtest.** Evaluating every calendar month with a 3-month-forward label would make consecutive months' `fwd_ret` overlap by 2 of 3 months — compounding those as a sequential return series (as `backtest.performance_summary` does) would badly overstate Sharpe/drawdown. Fixed by setting `test_months=1`, `step_months=HORIZON` in `run_walk_forward`, so evaluation only takes one snapshot every 3 months and each quarter's return window is adjacent to, not overlapping, the next. `freq=4` (not 12) is passed to `backtest.compare_models` to annualize correctly.
- **Information Coefficient (Spearman rank correlation), not R^2 ,** is the primary evaluation metric, since it evaluates rank order — what actually matters for a long-short portfolio built on ranks — rather than magnitude.
 - **Winsorization is cross-sectional (per month) and asymmetric in scope.** `features.winsorize` clips each feature to its 1st/99th percentile *within that month's cross-section* before z-scoring (`features.py`), so a handful of outliers don't distort the mean/std used to normalize every other stock that month. Separately, `model.py` winsorizes the *training* target only, per month, inside each walk-forward window — evaluation (IC, backtest returns) always uses the true, unwinsorized `fwd_ret`, so robustness applied to fitting never leaks into reported performance. This was tried specifically to address a near-zero IC; it didn't fix it (see Results) — reported as a negative result rather than dropped silently.

Known limitations (disclosed, not oversights)

- **No fundamentals data.** Factors are price/volume-based only (momentum, realized volatility, size, liquidity). A fundamentals- based value or quality factor is a natural extension, but requires as-reported (not fiscal-period-end) timing to avoid look-ahead leakage.
- **Transaction costs are not modeled directly.** Turnover is reported explicitly instead, as the input needed to estimate cost impact.

Pipeline

1. `data_prep.py` — pulls point-in-time S&P 500 membership and CRSP daily prices from WRDS, caches both to parquet
2. `features.py` — resamples to monthly, builds momentum/vol/size/ liquidity factors from full price history, applies the point-in-time membership filter, cross-sectionally z-scores, builds the forward-return target
3. `model.py` — walk-forward (rolling-window) cross-validation; trains a linear baseline and a LightGBM model on each window; scores predictions using the Information Coefficient (Spearman rank correlation)
4. `backtest.py` — builds a long top-decile / short bottom-decile portfolio from each model's predictions; computes annualized return, volatility, Sharpe, max drawdown, and turnover; compares models side-by-side
5. `main.py` — runs the full pipeline end to end

Running this project

```
pip install pandas numpy scikit-learn scipy lightgbm wrds pyarrow

python src/data_prep.py <wrds_username>    # one-time WRDS pull
python src/main.py                        # features -> models -> backtest
-> comparison
```

`data_prep.py` requires a WRDS account and will prompt for a password on first run if one isn't cached. WRDS has restructured table naming before (the newer CIZ-format tables are `crsp.dsf_v2` / `crsp.stksecurityinfohist`), so the pipeline verifies the expected tables and columns (`mbrstartdt`, `mbrenddt`, `permno`, `prc`, `ret`, `vol`, `shrout`, `cfacpr`, `cfacshr`) against the connected WRDS instance before pulling data.

What the results answer

The core question this project is built to answer isn't "did the strategy make money," but a calibrated comparison: does the nonlinear (LightGBM) model improve Information Coefficient and Sharpe ratio over the linear baseline, and at what turnover cost? That trade-off — stated plainly, alongside the point-in-time universe construction that avoids overstating the result — is the intended takeaway, and is what distinguishes a defensible backtest from a leaked one.

Possible extensions

- Fundamentals-based factors (value, quality) with as-reported timing to avoid look-ahead leakage
- Explicit transaction cost model applied to the turnover series